

1 Method

1.1 spike detection

The signal is first bandpass filtered (**300 – 6000 Hz or 100 – 6000 Hz**) through the MATLAB function `bandpass`. We detect neural spikes using a previously reported MATLAB implementation of an algorithm developed by Le Floch et al. (2022). By the algorithm, any timepoint is classified as a spike peak if its signal amplitude is lower than **-5** times of the signal standard deviation. A spike waveform is defined as a **4**-milliseconds-long slice of the signal centered at a spike peak.

1.2 burst detection

Spike bursts are detected using MaxInterval method Fair et al. (2020). It defines bursts using five pre-defined threshold parameters—maximum interspike interval at start of burst, maximum interspike interval in burst, minimum burst duration, minimum interburst interval, and minimum number of spikes in burst. Any series of spikes that satisfies all of these thresholds is considered as a burst. We set the threshold as **300, 301, 10, 50, 200**, respectively.

1.3 spike sorting through PCA and k-means (0.4, 0.5 in `summary_examples_multichannel.mlx`)

In order to sort the detected spike waveforms that originates from different neurons, we apply dimension reduction and clustering algorithms. We first use principal component analysis (MATLAB function `pca`) to reduce each high-dimensional spike waveform into vectors residing in a **two**-dimensional feature space. These vectors are then clustered into pre-specified number of groups, using the k-means algorithm (MATLAB function `kmeans`). We set the number of groups as **3**.

1.4 Spike sorting quality metrics

We adopt two measurements, namely L-ratio and isolation distance, to assess how each spike cluster is well-separated from the others. Both measurements build on Mahalanobis distance, which accounts for the correlation between dimensions when measuring the distance between a cluster and a spike. On the **two**-dimensional feature space of spikes, the squared Mahalanobis

distance between a cluster C and a spike x_i is defined as follows:

$$D_{i,C}^2 := (x_i - \mu_C)^\top \Sigma_C^{-1} (x_i - \mu_C),$$

where μ_C and Σ_C are the mean vector and the covariance matrix of the spike vectors belonging to cluster C , respectively. L-ratio assumes that the spike distribution if the cluster C follows a multivariate normal distribution, and is calculated as follows:

$$L_{ratio} := \frac{\sum_{i \notin C} (1 - CDF_{\chi^2}(D_{i,C}^2))}{n_C},$$

where $i \notin C$ indicates the set of spikes which do not belong the cluster C , CDF_{χ^2} represents the cumulative distribution function of the chi-square distribution with degree of freedom = 2, [jongmin: degree of freedom should is equal to the number of principal components.](#) and n_C indicates the number of cluster belonging to the cluster C . Isolation distance, on the other hand, is defined as the Malhalanobis distance of the n_C -th closest out-of-cluster spike.

1.5 3d network graph

Follow the approach of Lam et al. (2019), we quantify the synchronization level between two electrodes using the SPIKE-distance (Kreuz et al., 2013) between the spike trains. For the calculation of SPIKE-distance, we use Python package PYSPIKE (version 0.7.0) Mulansky and Kreuz (2016) on a Python environment (version 3.9.2). [The SPIKE-distance is normalized to average of 1000 randomly generated baseline values; each baseline value is the SPIKE-distance between two uniformly randomly generated spike trains.](#) Then, synchronization score is defined as SPIKE-distance values subtracted from 1. This score ranges from 0 to 1, where a higher score indicates a high level of synchrony.

Based on this synchronization score, community structure on the electrodes is examined by graph-theoretic Louvain algorithm (Newman and Girvan, 2004; Blondel et al., 2008) which is based on modularity optimization. In the analysis, we model the electrodes as nodes (e.g., circles in the network map), whose 3d positions represent the position of the actual electrode. We set the synchronized scores between every pair of active electrodes as edge (e.g., lines in the network map) weights between the corresponding nodes. Edges with synchronized scores less than 0.5 were filtered out. The result of the Louvain algorithm is visualized through color and size, using

the Python package `plotly` (version 5.9.0). Nodes with the same color (i.e., electrodes) represent electrodes belonging to the same community network. Synchronization scores are color-mapped, ranging from 0 (blue) to 1 (red). In addition, the node size indicates the number of connected electrodes.

References

- Blondel, V. D., Guillaume, J.-L., Lambiotte, R., and Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008.
- Fair, S. R., Julian, D., Hartlaub, A. M., Pusuluri, S. T., Malik, G., Summerfield, T. L., Zhao, G., Hester, A. B., Ackerman, W. E., Hollingsworth, E. W., Ali, M., McElroy, C. A., Buhimschi, I. A., Imitola, J., Maitre, N. L., Bedrosian, T. A., and Hester, M. E. (2020). Electrophysiological Maturation of Cerebral Organoids Correlates with Dynamic Morphological and Cellular Development. *Stem Cell Reports*, 15(4):855–868. Publisher: Elsevier.
- Kreuz, T., Chicharro, D., Houghton, C., Andrzejak, R. G., and Mormann, F. (2013). Monitoring spike train synchrony. *Journal of Neurophysiology*, 109(5):1457–1472.
- Lam, D., Enright, H. A., Cadena, J., Peters, S. K. G., Sales, A. P., Osburn, J. J., Soscia, D. A., Kulp, K. S., Wheeler, E. K., and Fischer, N. O. (2019). Tissue-specific extracellular matrix accelerates the formation of neural networks and communities in a neuron-glia co-culture on a multi-electrode array. *Scientific Reports*, 9(1):4159. Number: 1 Publisher: Nature Publishing Group.
- Le Floch, P., Li, Q., Lin, Z., Zhao, S., Liu, R., Tasnim, K., Jiang, H., and Liu, J. (2022). Stretchable Mesh Nanoelectronics for Three-Dimensional Single-Cell Chronic Electrophysiology from Developing Brain Organoids. *Advanced materials (Deerfield Beach, Fla.)*, 34(11):e2106829.
- Mulansky, M. and Kreuz, T. (2016). PySpike—A Python library for analyzing spike train synchrony. *SoftwareX*, 5:183–189.
- Newman, M. E. J. and Girvan, M. (2004). Finding and evaluating community structure in networks. *Phys. Rev. E*, 69:026113.